

# Build Your First Agent Workflow

A real task. A boundary. A result you can check.

Heather Wilde · @heathriel  
KCDC 2026 · September 9 · Room 2215-A

[github.com/heathriel/first-agent-workshop](https://github.com/heathriel/first-agent-workshop)

**BUILD / REMEMBER / REUSE / VERIFY**

# Hi, I'm Heather.



- Early employee, Evernote & Spirit Airlines
- AI field engineer and CTO
- Writer, advisor, and builder of useful systems
- Author of Birth of a Unicorn
- I run agents in production. I also clean up after them.

OPEN THE  
REPO  
Follow along  
[@heathriel](#)

[@heathriel](#)

# Get one small thing working

- 1 · Open the downloaded workshop folder (not the GitHub webpage)
- 2 · Start your signed-in agent in the repository folder
- 3 · Run: `python3 workshop.py preflight` (Windows: `py -3`)
- 4 · Ask it to create `output/hello.md`, then read it back

**Ready? Help the person next to you. You have been promoted.**

*Blocked? Pair up or use [cheatsheets/universal.md](#). Follow your organization's network policy.*

Let's see the finished result.  
Then we'll build the pieces.

# Five additions to one working task

|                           |                             |       |                                                 |
|---------------------------|-----------------------------|-------|-------------------------------------------------|
| <b>1 · First agent</b>    | a goal + a boundary         | 8:20  |                                                 |
| <b>2 · Memory</b>         | it remembers Tuesday        | 8:55  | Breaks: 9:10 and 10:30<br>Ten minutes each.     |
| <b>3 · Skills + tools</b> | taught, then connected      | 9:20  | Every module leaves<br>evidence.                |
| <b>4 · Loops</b>          | checks, retries, then stops | 10:00 | Core finish: 11:30<br>Optional help until noon. |
| <b>5 · A team</b>         | workers + a reviewer        | 10:40 |                                                 |

# Find a job worth handing over

Tell your partner:

**What did you do twice last month? Walk your partner through it.**

Round 2:

**What must stay under your control? Make that boundary specific.**

**Choose a small, checkable task. Stuck? Use the fictional meeting in `data/meeting.txt`.**

## The model proposes. The runtime decides.

```
for attempt in range(3):  
    proposal = model(goal, context)  
    result = permitted_tool(proposal)  
    if check(result):  
        break  
else: report_failure()
```

# Give it a job that fits

## Brilliant

Useful at reading, drafting, and tool selection

## Context-dependent

A fresh session needs the right information loaded

## Jagged

Strong in one task; unreliable in the next

## Does your task benefit from an agent?

- Fixed procedure → start with a script or workflow.
- Variable steps + an evaluable outcome → try an agent.
- Unclear outcome → narrow the task or keep human judgment.

# Build the first version · 20 minutes

- 1 · Open `JOB_DESCRIPTION.md` locally; keep your task separate
- 2 · Specify the output, one boundary, and the proof before running
- 3 · See the bad fixture **FAIL**; keep the checker unchanged
- 4 · Run your prompt. Save `output/summary.json`. Check it.
- 5 · Ask it to invent the missing owner. Record what actually happens.

**Evidence: checked output + boundary test. A refusal is an observation, not a security proof.**

# A successful run can still miss the job.

- The extraction check is green.
- The user asked: Can we launch today?
- Write the answer they actually need.
- What acceptance criterion did our check miss?

**PASS**  
**≠ DONE**

A new session needs a handover.

# Keep the handbook small and test it.

- AGENTS.md: project rules and checks
- CLAUDE.md: imports @AGENTS.md and @memory.md
- memory.md: a few dated, relevant facts
- File discovery depends on the tool.

Keep information the next run needs.  
Correct stale facts; leave secrets out.

## What the research actually supports

*A 2026 coding-agent study found no general success gain from context files.*

- Average inference cost rose by over 20% in the evaluated tasks.
- That is not proof that every handbook is harmful. Test yours.

# Save it. Close it. Recover it. · 10-minute lab

1 · Create AGENTS.md; Claude users import it through CLAUDE.md

2 · Save a fictional fact + one preference in memory.md

3 · Start a fresh session in the same folder. Do not resume.

4 · Ask for the fact and preference. Inspect the files it loaded.

**Evidence: fresh-session recovery, then a saved correction to a stale fact.**

Break. Ten minutes.

Back at 9:20.

*Next: turn a useful procedure into a reusable skill.*

A skill is a procedure you can reuse.  
Give it a clear trigger and a test.

# Write down the mistake you won't repeat.

Mine, for example:

**“NEVER introduce a name that isn't in the transcript.”**

One invented name was enough to damage trust.  
Now I test the missing-owner case deliberately.

**Before you install someone else's skill**

- Read the instructions and bundled scripts.
- Check the source and the permissions it needs.
- Treat updates as changes worth reviewing.
- Writing your own does not remove the need to test.

# Make a choice. Show the evidence. · 25 minutes

**A · Create `.claude/skills/meeting-summary/SKILL.md` (or your tool's equivalent)**

A · Add trigger, steps, output format, and one missing-information rule

**B · Predict which follow-ups need the project directory. Then run.**

B · Resolve a role. Preserve an unknown. Expose a conflict.

**Evidence: `output/followups.json` + actual tool-read result + a human question.**

*Use `modules/03-skills/DECISION_LAB.md`. MCP setup is optional homework.*

# ARRR, MATEY?

**Did the document give your agent an order?**

- Summarize the fictional report. Observe what actually happens.
- If it follows the hidden instruction: an injection succeeded.
- If it ignores it: this attempt failed. Keep that result.
- Untrusted content stays data. Tools and approvals enforce the boundary.

## **A retry loop needs an exit.**

Check each attempt. Stop on success or a limit.

A schedule decides when to start; a controller decides when to stop.

# The limit belongs outside the model.

**\$20.50**

My agent tried to exceed its \$20/day budget by fifty cents.  
The cap blocked the overrun and surfaced the failure.

Best fifty cents I never spent.

## Controls to inspect:

- **Unchanged checker after each attempt**
- **Three attempts, then a visible failure report**
- **Time and usage limits in the runtime**
- **A named human owns the next decision**

# Make it stop · 20-minute lab + debrief

1 · Fix the criterion first. Run `python3 workshop.py rehearse`

2 · Live: produce → check → return errors. Maximum three attempts.

3 · Run `python3 workshop.py rehearse --impossible` (exit 2 is expected)

4 · Inspect the escalation. Who owns the missing-information decision?

**Evidence: DONE on a passing result; ESCALATED on the impossible path.**

*Optional: `python3 live_loop.py`. Needs Claude CLI + API key; billed separately.*

Break. Ten minutes.

Back at 10:40.

*Next: two workers, then a reviewer who sees their work.*

Add a team when the work separates.  
Keep the single-agent baseline.

# A reviewer needs evidence, not a personality.

*A different role can help. It does not guarantee independent judgment.*

- Workers can propagate the same mistake.
- Give the reviewer the original source.
- Wait for both workers before review.
- Cap the team's time, output, and fan-out.

**Research: more agents is not always better**

A controlled study of 180 configurations found benefits varied by task and coordination.

Parallel work can benefit. Sequential work can suffer. Compare your own outcomes.

# Two workers. Then the reviewer. · 18-minute lab

- 1 · Open `ROLE_PROMPTS.md`; four ordinary chats are enough
- 2 · A checks coverage. B checks unsupported commitments. Wait for both.
- 3 · Reviewer gets both outputs AND the source. Test the weakest claims.
- 4 · Lead resolves objections with evidence. A human checks the result.

**Evidence:** four saved outputs; Alex corrected to UNKNOWN; source lines cited.

# Change the input. Does the lesson survive?

**Choose F2 or F4**

Remove or contradict the role mapping on paper.

**Predict before running**

What should the agent do with the missing evidence?

**State the human question**

Who can resolve the owner or source conflict?

**Challenge the checker**

Which fixed expectation is now invalid, and why?

**Keep the boundary**

No invented owner. No approval sent.

**Show your reasoning**

A changed contract needs a justified test change.

# Design your own next run. Then try to break it.

- 1 Name the user, decision, input, and smallest useful output.
- 2 Identify a real choice. A script may be the right answer.
- 3 **Allow one action. Withhold one. Name the stop question.**
- 4 Write a passing case and a plausible-looking failure.
- 5 Partner: remove a fact. Does the plan still behave honestly?

I built my agent a mailbox.  
He ignored it for two weeks.

Eventually it started using it.

My lesson: build for a need you can observe.  
Then inspect what the system actually does.

# Run it again tomorrow.

You have a task, context, a procedure, and checks.  
Keep the evidence. Improve the part that actually fails.

**Share-out: job · boundary · check · next run**

**What does it do? What can't it do? How will you know?**

**Continue the conversation: “I Fight for the User” with Matthew, Bob, and me.  
See the conference schedule for the session details.**

**[github.com/heathriel/first-agent-workshop](https://github.com/heathriel/first-agent-workshop) · questions and fixes welcome**